# Architecture and Implementation of a Personalized Digital Handwriting Synthesis and Document Rendering Engine

The transition from digital document creation to analog submission represents a persistent friction point in modern academic, professional, and creative workflows. While digital text generation—augmented by advanced word processors, Markdown editors, and artificial intelligence—offers unprecedented speed and structuring capabilities, many institutions and personal workflows still mandate physical, handwritten submissions. Conventional tools attempting to bridge this divide rely on static handwriting fonts or basic PDF converters. These tools invariably fail the Turing test of visual authenticity. Standard handwriting fonts rely on rigid, pixel-identical glyphs that lack the biomechanical variance, spatial drift, and kinematic ink flow inherent to human writing. Furthermore, mathematical notations, code blocks, and diagrams frequently break standard font converters, exposing raw LaTeX syntax or resulting in disjointed, unreadable equations.

To resolve these limitations, this report outlines the comprehensive architecture of a Personalized Digital Handwriting Synthesis and Document Rendering Engine. Elevating the concept from a simplistic visual hack to a robust, dual-use platform, this architecture integrates sequence-based machine learning models, biomechanical kinematic algorithms, Abstract Syntax Tree (AST) parsing, and computer vision-based spatial mapping. The resulting system possesses massive real-world utility, serving not only students but also neurodivergent individuals struggling with dysgraphia or repetitive strain injuries (RSI), graphic designers requiring authentic organic textures, and professionals generating personalized digital correspondence.

## 1. The Core Synthesis Engine: Morphological and Biomechanical Modeling

True handwriting realism is achieved not through precise mechanical replication, but through controlled, algorithmic inconsistency. Human handwriting is the product of a complex biomechanical system subject to physiological noise, muscular fatigue, and neuro-motor planning. The core synthesis engine must transition away from static typography and instead model these variations dynamically across multiple axes of generation.

### 1.1 The Multi-Sample Glyph Vault and Zero-Shot Style Extraction

Standard typography maps a single vector image to a specific character encoding. To prevent the output from appearing robotic, the engine abandons this one-to-one mapping in favor of a Multi-Sample Glyph Vault.

In a standard calibration phase, the system prompts the user to input three to five variations of each character (e.g., how a lowercase 's' is formed at the beginning of a word versus the end). However, to satisfy the requirement for an advanced, frictionless user experience, the system implements a computer vision pipeline capable of zero-shot and few-shot style extraction directly from existing photographs of the user's past work.

This architectural pipeline utilizes state-of-the-art Offline Handwritten Text Generation (HTG) techniques. Early approaches to handwriting synthesis utilized Long Short-Term Memory (LSTM) recurrent neural networks to generate sequences of pen coordinates, predicting the relative $(x, y)$ offsets and pen-state transitions[1]. While effective for online generation where temporal stroke data is available, pure RNN models struggle with static image conditioning and require extensive trajectory training data[3].

Modern implementations leverage Diffusion Models and Vision Transformers (ViT) to extract style representations from a minimal set of reference images. Frameworks such as DiffusionPen and CONSTANT (Patch Contrastive Enhancement and Style-Aware Quantization) operate effectively as few-shot generators[4]. By processing a photograph of a user's previous assignment, the system utilizes a Style-Aware Quantization (SAQ) module to capture invariant features like slant, stroke width, character spacing, and curvature[6]. The photograph is first binarized, and individual words are segmented using algorithms like Character Region Awareness for Text Detection (CRAFT)[8]. The Vision Transformer then tokenizes these visual patches, learning the global stylistic patterns and building a synthetic multi-sample glyph vault automatically, without requiring the user to trace characters on a tablet[6].

| Architecture Paradigm | Mechanism of Action | Style Adaptation Capability | Primary Architectural Trade-off |
|---|---|---|---|
| **LSTM-RNN (Graves, 2013)** | Predicts sequential $(x, y)$ pen coordinates and pen-up/down states based on target text strings[2]. | Requires extensive temporal trajectory data; highly difficult to adapt to static photographic images[3]. | High computational cost for long-range dependencies; lacks robust static image conditioning capabilities. |
| **Transformer (HWT)** | Self-attention mechanism encoding both global and local style patterns from reference images[7]. | Few-shot setting; learns complex style-content entanglement efficiently[9]. | Requires significant memory allocation for self-attention over extended text lines and paragraphs. |

| | | | |
|---|---|---|---|
| **Diffusion (DiffusionPen)** | Metric learning combined with denoising diffusion for stylized offline rendering[4]. | One-shot / Few-shot; captures unseen styles effectively from minimal samples[4]. | High inference latency due to iterative denoising steps, requiring optimization for real-time web rendering. |
| **CONSTANT (ViT + SAQ)** | Extracts discrete visual tokens of style via latent patch contrastive enhancement[6]. | Zero-shot / One-shot; highly detailed image generation isolated from background noise[6]. | Complex latent space optimization required during training; high baseline resource requirements. |

## 1.2 Ligature and Connection Modeling

Handwriting is a continuous flow, and the geometry of a character is heavily influenced by its neighbors. A lowercase 't' followed by an 'h' has a distinct connecting stroke that differs fundamentally from a 't' followed by an 'o'.

To model this, the rendering engine utilizes a Markov-chain probabilistic system combined with a rule-based ligature engine. When synthesizing a word, the system analyzes character bigrams and trigrams. If the user's extracted style exhibits cursive or semi-cursive traits, the engine calculates the exit trajectory vector of the preceding character and the optimal entry coordinate of the succeeding character. It then generates a connecting spline that bridges these points. The Markov-chain system evaluates the probability of a stroke break versus a continuous ligature based on the extracted style parameters, ensuring that the rhythm of the connected text matches the user's natural cadence.

## 1.3 The Natural Imperfections Layer and the Variance Threshold

True realism relies on controlled inconsistency. Human handwriting is inherently flawed, but these flaws operate within specific biomechanical constraints. If the baseline drift or glyph randomness is set too high, the output resembles messy digital warping or algorithmic failure. When kept strictly within a 5–15% variance threshold, human visual perception accepts the output as naturally occurring handwriting.

To achieve this, the engine applies multiple layers of structural perturbation:

1. **Baseline Drift:** Human handwriting never sits on a mathematically laser-straight horizontal line. The engine introduces a gentle, low-frequency compound sine wave algorithm to the horizontal baseline of each rendered line. This causes words and entire sentences to drift slightly upward or downward across the page, simulating the natural arc of a wrist pivoting around a fixed elbow resting on a desk.
2. **Variable Kerning and Pacing:** Humans naturally leave slightly wider gaps after commas, periods, or when pausing for cognitive processing. The engine adds randomized

micro-spacing between words and punctuation. Rather than a fixed space width, the gap is determined by a Gaussian distribution scaled by the user's average spacing metrics extracted during the calibration phase.

## 1.4 Kinematic Realism via the Two-Thirds Power Law

A highly visible flaw in basic digital ink rendering is uniform stroke thickness and constant opacity. Human writing instruments interact with physical paper dynamically, driven by the velocity and pressure of the stroke. This relationship is mathematically governed by the Two-Thirds Power Law of human motor control, which describes the invariant relationship between the angular velocity of a movement and the curvature of its trajectory[10].
The law dictates that as the curvature of a trajectory increases, the human hand must systematically slow down[11]. Expressed mathematically, the angular velocity $\omega$ is proportional to the two-thirds power of the curvature $\kappa$:

$$\omega = K \cdot \kappa^{2/3}$$

Alternatively, expressed in terms of tangential velocity $V$ and the radius of curvature $R$:

$$V = K \cdot R^{1/3}$$

In the context of the synthesis engine, this biomechanical law is weaponized to simulate pen pressure and ink dynamics. As a generated stroke enters a high-curvature phase—such as the tight loop of an 'l' or the sharp peak of an 'i'—the calculated theoretical velocity drops. The rendering engine correlates this velocity drop with an increase in simulated hand pressure. Consequently, the HTML5 Canvas rendering context dynamically modulates the lineWidth and globalAlpha parameters. Slower, high-curvature downstrokes are rendered slightly thicker and darker, simulating wet-ink saturation or heavy graphite deposition[12]. Conversely, low-curvature, high-speed connecting strokes between letters are rendered thinner and slightly lighter, simulating the rapid glide of the pen across the paper's surface.

## 1.5 Smooth Path Generation via Parametric Bézier Curves

When extracting strokes from user photographs, traditional pixel-based skeletonization methods often produce fragmented, jagged strokes that fail to preserve accurate topological structures[13]. To guarantee infinitely scalable and mathematically smooth output, the engine models all handwriting as parametric cubic Bézier curves rather than raster arrays.
The Canvas 2D API utilizes the bezierCurveTo(cp1x, cp1y, cp2x, cp2y, x, y) function, which constructs a curve defined by a start point, two control points, and an end point[14]. The cubic

Bézier function $B(t)$ for a parameter $t$ ranging from 0 to 1 is computed as:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t)t^2 P_2 + t^3 P_3$$

By deploying deep learning-based image registration, the engine aligns standard vector reference paths with the synthetic raster output generated by the Diffusion or Transformer models. It subsequently applies a distance-minimization algorithm to fit the Bézier control points directly onto the target character's skeleton[13]. This ensures that the generated vector ink remains fluid, allowing for real-time interpolation and dynamic scaling without pixelation artifacts.

# 2. Interactive Calibration and the Sandbox Environment

To visualize how these biomechanical layers transform rigid digital text into natural handwriting, the platform requires an interactive sandbox calibration environment. During initial onboarding, the user is presented with a real-time drawing setup via a touchscreen interface or tablet stylus. This serves a dual purpose: gathering high-fidelity temporal data (if the user opts for manual calibration rather than photographic upload) and establishing a visual baseline.
The interactive comparison tool allows users to toggle individual parameters in real-time. A slider controls the 5–15% variance threshold, providing immediate visual feedback on how baseline drift, kerning randomization, and the Two-Thirds Power Law affect the output. This sandbox ensures that users can fine-tune the engine to model their specific physical handwriting nuances accurately, transitioning the system from a black-box AI generator into a deterministic, user-controlled rendering suite.

# 3. Native LaTeX, Code Block, and Algorithmic Diagram Rendering

A catastrophic failure point in existing document converters is the handling of structured data, programming code, and mathematical equations. If an AI agent or a user generates a LaTeX formula, a naive backend will attempt to render the raw string (e.g., \frac{-b \pm \sqrt{b^2-4ac}}{2a}) directly into a handwriting font, resulting in illegible syntax. The backend must parse these structures cleanly.

## 3.1 Abstract Syntax Tree (AST) Interception and SVG Vectorization

To synthesize realistic mathematics, the system intercepts specific block identifiers (like $$ or \begin{equation}) and routes them through a dedicated parsing layer utilizing libraries such as MathJax or KaTeX[15].
The pipeline for mathematical rendering operates through a series of discrete transformations:

1. **AST Generation:** The parser ingests the LaTeX string and constructs an internal MathML

Abstract Syntax Tree (AST), which semantically maps the spatial relationships of numerators, denominators, integrals, and limits[16].

2. **SVG Path Decomposition:** Rather than relying on HTML/CSS layouts and rigid font files, the AST is explicitly rendered into Scalable Vector Graphics (SVG) paths[15]. This bypasses web-font rendering issues entirely, yielding pure geometric coordinate data for every element in the equation[15].
3. **Handwritten Symbol Mapping:** The standard <path> elements output by MathJax consist of typographically perfect, machine-drawn outlines[17]. The synthesis engine intercepts these SVG nodes, identifies their semantic meaning (e.g., recognizing the integral symbol $\int$ or the summation symbol $\sum$), and replaces the rigid vector path with the user's custom, parameterized handwritten equivalent.
4. **Fallback Glyph Injection:** If the user has not recorded specific complex symbols (e.g., Greek letters like $\alpha, \beta$, or matrix brackets), the system automatically supplies a default "neat student handwriting" fallback glyph set that matches the stroke width of the user's primary writing style[18].

### 3.2 Code Block Handling and Monospace Simulation

For computer science assignments, the platform must differentiate between prose and programming syntax. The engine detects Markdown code blocks (delimited by triple backticks) and automatically shifts its generation parameters. When simulating how a programmer writes code on paper, the engine enforces a slightly more structured, monospaced handwritten style. Ligatures are disabled, vertical alignment is strictly enforced to preserve indentation (critical for languages like Python), and character variance is reduced to mimic deliberate, block-letter printing.

### 3.3 Algorithmic Flowchart and Diagram Sketching

Assignments frequently contain structural diagrams, flowcharts, or ASCII art. Rendering a pristine, mathematically perfect vector graphic alongside organic human handwriting breaks the visual illusion of a fully physical document.

To bridge this aesthetic gap, the pipeline integrates algorithmic drawing libraries such as Rough.js[19]. When the system detects Mermaid.js syntax, state diagrams, or Markdown tables, it routes the structural geometry through this sketching engine. Rough.js interprets geometric primitives (lines, polygons, ellipses, paths) and redraws them with a sketchy, hand-drawn appearance[19]. The library applies intentional overshooting at vertices, subtle curvature to straight lines, and variable stroke widths, outputting an SVG that perfectly matches the surrounding handwritten text.

## 4. Context-Aware Continuation: Bridging Physical and Digital Spaces

To elevate the platform into a daily-use productivity suite, it must interact seamlessly with the physical world. A high-value capability is "Smart Margin & Alignment OCR." This allows users to

upload a photograph of a partially filled physical notebook page, and the engine seamlessly appends new digital handwriting exactly where the physical pen stopped.

## 4.1 Document Rectification and Lighting Analysis

Ingesting a physical photograph requires a robust computer vision pipeline to correct environmental distortions:

1. **Homography and Perspective Warping:** The uploaded image is analyzed to detect the four corners of the physical page bounding box. A perspective transform (homography) matrix is calculated and applied to flatten the image, correcting any skew or keystone effects introduced by the angle of the smartphone camera.
2. **Lighting Normalization:** The system calculates the background illumination gradient. It identifies uneven lighting, shadows from the user's hand, and color temperature variations across the paper's surface, preserving these features for later blending.

## 4.2 Hough Line Transforms and Projection Profiles

To accurately place digital text onto a lined physical notebook, the system must detect both the printed margins and the existing handwriting.

**Physical Line Detection:** The OpenCV Hough Line Transform algorithm is deployed to detect the college-ruled or wide-ruled lines on the paper[21]. The algorithm operates by transforming the image space into a polar coordinate parameter space defined by $(\rho, \theta)$, where $\rho$ is the distance from the origin and $\theta$ is the angle[21]. This mathematical transformation allows the system to detect continuous horizontal lines robustly, even if the lines are faint, fragmented, or slightly rotated due to imperfect document rectification[21].

**Text Line Segmentation:** To isolate the user's existing handwriting from the blank space, the system utilizes a Horizontal Projection Profile (HPP)[23]. The image is converted into a binary matrix, and the HPP calculates the sum of the intensity of black pixels along each horizontal row of the image array[23].

The resulting HPP generates a topological histogram. High peaks in the histogram indicate the dense presence of text lines, while deep valleys (approaching zero height) indicate the white space or line spacing between paragraphs[24]. By algorithmically identifying the final peak in the projection profile, the engine pinpoints the exact pixel coordinate where the physical writing ceases. The synthesis engine then anchors the starting coordinate for the newly generated digital text to the subsequent ruled line identified by the Hough Transform, ensuring pixel-perfect continuity.

# 5. Advanced Realism Studio and Multi-Instrument Modeling

To ensure the exported document is indistinguishable from a photograph of a physical assignment, the platform includes a Paper and Lighting Realism Studio. Exporting a flat, flawless PDF with a pure white background instantly reveals the document's digital origins.

## 5.1 Multi-Instrument Physics Simulation

Different writing tools exhibit unique physics when interacting with paper. The engine offers parameterized models for various instruments, manipulating the HTML5 Canvas blending modes and Bézier rendering algorithms to simulate specific physical properties:

| Instrument Type | Edge Sharpness | Ink Opacity Dynamics | Canvas Blending Mode / Special Effects |
|---|---|---|---|
| **Ballpoint Pen** | High / Sharp | Variable based on velocity; occasional micro-skipping on rapid upstrokes. | multiply blend mode; hard edges with dark, concentrated core strokes. |
| **Gel Pen** | Low / Bleed | High, uniform saturation; minimal velocity dependence. | darken; simulates wet-ink bleed into the paper fibers; slight blooming effect. |
| **Mechanical Pencil** | Medium | Highly pressure-dependent; overall low opacity (grey tones). | source-over with noise textures; generates a subtle graphite sheen and texture grain. |
| **Highlighter** | Very Low | Semi-transparent; uniform width. | multiply with neon color palettes; layered beneath the primary text strokes. |

## 5.2 Environmental Realism and 3D Warping

The final rendering pass applies environmental realism to the document. Rather than a 2D export, the canvas is passed through a WebGL rendering context[26].

- **3D Perspective Warping:** The WebGL pipeline applies a subtle 3D mesh warp to the document, introducing gentle page curvature that simulates the binding of a notebook or the natural resting state of paper.
- **Fragment Shaders and Texture Overlays:** Dynamic shadow gradients are overlaid to

simulate a desk lamp or ambient room lighting. Background texture maps inject imperfections such as coffee cup ring stains, eraser smudges, and simulated smartphone camera grain. The final output file mirrors the exact aesthetic of a high-resolution photograph taken of a physical notebook resting on a desk.

# 6. Privacy, Safety, and Ethical Guardrails

Handwriting is a distinct, deeply personal, and legally binding biometric identifier[27]. The mass collection of handwriting samples poses severe privacy and security risks. If users suspect their biometric data or private university assignments are being harvested for training datasets, or worse, leaked to malicious actors for contract spoofing or digital check washing, adoption will instantly collapse[28]. Therefore, the architecture of this synthesis engine is designed around absolute cryptographic isolation and a privacy-first paradigm.

## 6.1 Zero-Knowledge Architecture via WebAssembly

To guarantee biometric privacy, the entire processing pipeline is executed strictly on the client side. The deep learning inference models responsible for style extraction (Vision Transformers) and sequence generation are quantized and compiled into WebAssembly (Wasm)[29]. By leveraging the WebGPU API, these models execute entirely within the local environment of the user's browser, utilizing the local GPU for hardware acceleration[29].

The user's biometric glyph variations, photographic references, and custom datasets are encrypted and stored solely in the browser's local IndexedDB vault. The system guarantees that neither the source material nor the generated biometric ink ever traverses an external network. Documents and biometric data never touch a cloud server, immunizing the platform against third-party scraping or centralized database breaches.

## 6.2 Ethical Micro-Watermarking and Forensic Traceability

Dual-use technologies such as handwriting synthesis carry inherent risks of malicious application, such as forging authorizing signatures on financial documents. To neutralize this threat, the rendering engine embeds ethical micro-watermarking directly into the exported document.

1. **Spatial and Frequency Steganography:** A proprietary steganographic algorithm subtly alters the spatial distances between specific strokes or embeds high-frequency noise patterns into the rasterized PDF export[30]. This watermark is entirely imperceptible to the human eye, maintaining the clean aesthetic required for academic notes, but is instantly readable by detection algorithms.
2. **Defeating TS-GATR Verification:** Forensic verification of dynamic signatures relies heavily on identifying genuine temporal and spatial features. Modern detection systems, such as the Temporal-Spatial Graph Attention Transformer (TS-GATR), model signatures as graphs where nodes capture dynamic features like velocity, pressure, and temporal stroke order[31]. While the synthesis engine highly accurately models the visual output, it intentionally embeds specific, mathematically deterministic anomalies into the micro-structure of the exported vector paths. If a synthesized document is subjected to forensic analysis or TS-GATR verification, these anomalies flag the document as

synthetic, ensuring that generated signatures will fail biometric authentication and cannot be used for fraud[31].

Furthermore, during the Sandbox Calibration phase, the system requires a guided, real-time drawing setup if a user attempts to generate a verified signature block. This ensures that users can only model and utilize their own handwriting styles for high-security applications, preventing the uploading and spoofing of another individual's signature.

# 7. System Evaluation and Quality Assurance Metrics

To ensure the synthesis engine remains firmly within the acceptable variance threshold—avoiding mechanical rigidity while preventing illegible digital warping—the generated output is continuously evaluated against established Handwritten Text Recognition (HTR) benchmarks[33].

During internal system calibration, the generated documents are passed through a third-party HTR neural network[35]. The system evaluates the output using two primary metrics:

| Metric | Measurement Focus | Evaluation Mechanism | Optimal Threshold |
|---|---|---|---|
| **Character Error Rate (CER)** | Readability and Legibility | Measures the minimum number of character-level operations (insertions, deletions, substitutions) required to align the HTR transcription with the source digital text[36]. | Must remain <10%. Higher rates indicate the baseline drift or glyph randomness algorithms are distorting the text beyond human legibility[37]. |
| **Handwriting Distance (HWD)** | Stylistic Authenticity | Measures the perceptual distance of subtle geometric features between the generated synthetic images and the original physical reference handwriting using Euclidean distance | Must minimize distance. A low HWD score confirms the engine successfully captured the nuanced kinematic and morphological characteristics of the specific |

| | | | |
|---|---|---|---|
| | | in a specialized feature space[3]. | writer[38]. |

By dynamically balancing the CER for legibility and the HWD for stylistic authenticity, the synthesis engine continuously self-optimizes, ensuring that the final rendering represents the pinnacle of digital-to-analog document translation.

# Conclusion

The Personalized Digital Handwriting Synthesis and Document Rendering Engine detailed in this architectural report fundamentally redefines the intersection of digital efficiency and analog presentation. By synthesizing the biomechanical precision of the Two-Thirds Power Law, cubic Bézier morphological modeling, and advanced mathematical AST rendering, the platform produces documents indistinguishable from authentic human manuscripts. Furthermore, the integration of projection profile-based spatial mapping allows for unprecedented, context-aware interactions between physical photographs and digital algorithms, enabling seamless continuation of physical notes. Underpinned by a rigorous zero-knowledge WebAssembly architecture and robust steganographic anti-forgery watermarks, this system securely democratizes handwriting generation. It provides profound accessibility benefits for neurodivergent individuals and seamless, highly realistic automation for academic and professional environments, completely resolving the friction of the digital-to-analog divide.

**Works cited**


1. GitHub - swechhasingh/Handwriting-synthesis: Implementation of "Generating Sequences With Recurrent Neural Networks" https://arxiv.org/abs/1308.0850 · GitHub, https://github.com/swechhasingh/Handwriting-synthesis
2. sjvasquez/handwriting-synthesis - GitHub, https://github.com/sjvasquez/handwriting-synthesis
3. SpiS-GAN: Spiral-Modulated Handwriting Synthesis with Star Operation - arXiv, https://arxiv.org/html/2607.06949v1
4. Official PyTorch Implementation of "DiffusionPen: Towards Controlling the Style of Handwritten Text Generation" - ECCV 2024 · GitHub, https://github.com/koninik/DiffusionPen
5. CONSTANT: Towards High-Quality One-Shot Handwriting Generation with Patch Contrastive Enhancement and Style-Aware Quantization - arXiv, https://arxiv.org/html/2603.07543v1
6. [2603.07543] CONSTANT: Towards High-Quality One-Shot Handwriting Generation with Patch Contrastive Enhancement and Style-Aware Quantization - arXiv, https://arxiv.org/abs/2603.07543
7. [PDF] Handwriting Transformers | Semantic Scholar, https://www.semanticscholar.org/paper/Handwriting-Transformers-Bhunia-Khan/103f9830c3b7efff0233b5755c49da4818d2b8cf
8. A Case Study of Handwritten Text Recognition from Early Modern Sanskrit Manuscripts - ACL Anthology, https://aclanthology.org/2025.wsc-csdh.4.pdf

9. ankanbhunia/Handwriting-Transformers - GitHub, https://github.com/ankanbhunia/Handwriting-Transformers
10. The role of motor simulation in action perception: a neuropsychological case study - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC2694935/
11. Biological kinematics: a detailed review of the velocity-curvature power law calculation - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC11968483/
12. (PDF) Biological kinematics: a detailed review of the velocity-curvature power law calculation - ResearchGate, https://www.researchgate.net/publication/390466806_Biological_kinematics_a_detailed_review_of_the_velocity-curvature_power_law_calculation
13. Bézier Curve-Based Stroke Extraction for Handwritten Characters - BMVA Archive, https://bmva-archive.org.uk/bmvc/2025/assets/papers/Paper_478/paper.pdf
14. CanvasRenderingContext2D: bezierCurveTo() method - Web APIs - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D/bezierCurveTo
15. SVG Support — MathJax 4.0 documentation, https://docs.mathjax.org/en/v4.0/output/svg.html
16. MathJax Output Formats, https://docs.mathjax.org/en/v4.0/output/index.html
17. SVG output · Issue #375 · KaTeX/KaTeX - GitHub, https://github.com/KaTeX/KaTeX/issues/375
18. How to create non-outlined SVG files from LaTeX formulae - TeX, https://tex.stackexchange.com/questions/282340/how-to-create-non-outlined-svg-files-from-latex-formulae
19. Rough.js, https://roughjs.com/
20. Mermaid Rendering, Made Beautiful - Craft Agent, https://agents.craft.do/mermaid
21. https://data-flair.training/blogs/hough-line-transform/#:~:text=or%20missing%20segments.-,In%20computer%20vision%20and%20image%20processing%2C%20the%20OpenCV%20Hough%20Line,and%20distance%20from%20the%20origin.
22. Hough Transform for Line Detection - GitHub, https://github.com/AlbertoFormaggio1/Hough-Transform-From-Scratch
23. Projection Histogram of Binary Image - Sachith Sampath Priyashantha - Medium, https://sachithppp.medium.com/projection-histogram-of-binary-image-e4e9eabb7c9d
24. Optical Character Recognition (OCR) Using Opencv and Python: - IGI Global, https://www.igi-global.com/viewtitle.aspx?TitleId=350806&isxn=9798369333549
25. how to segment text/handwritten lines using horizontal profile projection? - Stack Overflow, https://stackoverflow.com/questions/73232684/how-to-segment-text-handwritten-lines-using-horizontal-profile-projection
26. WebGL: 2D and 3D graphics for the web - Web APIs - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API

27. Exploring Transformers for On-Line Handwritten Signature Verification - arXiv, https://arxiv.org/html/2307.01663
28. (PDF) Remote Check Truncation Systems: Vulnerability Analysis and Countermeasures, https://www.researchgate.net/publication/340123814_Remote_Check_Truncation_Systems_Vulnerability_Analysis_and_Countermeasures
29. WebGPU for Machine Learning: Running Models on the GPU | MinhVo, https://minhvo.is-a.dev/blogs/webgpu-for-machine-learning-running-models-on-the-gpu
30. WO2019246473A1 - Method of determining authenticity of a consumer good - Google Patents, https://patents.google.com/patent/WO2019246473A1/en
31. Online Handwritten Signature Verification Based on Temporal-Spatial Graph Attention Transformer - arXiv, https://arxiv.org/html/2510.19321v1
32. Online Handwritten Signature Verification Based on Temporal-Spatial Graph Attention Transformer - arXiv, https://arxiv.org/pdf/2510.19321
33. Handwritten Text Recognition: A Survey - IEEE Computer Society, https://www.computer.org/csdl/journal/tp/2026/04/11303599/2cygznd3hcY
34. Benchmarking Large Language Models for Handwritten Text Recognition - arXiv, https://arxiv.org/pdf/2503.15195
35. A study of handwritten text recognition with cross-out words - ResearchGate, https://www.researchgate.net/publication/408351639_A_study_of_handwritten_text_recognition_with_cross-out_words
36. Rethinking HTG Evaluation: Bridging Generation and Recognition - arXiv, https://arxiv.org/html/2409.02683v1
37. A Hybrid CNN-LSTM Model for Handwritten Text Recognition with Comparative Analysis of Similarity Algorithms for Automated Answer Scoring | IIETA, https://www.iieta.org/journals/isi/paper/10.18280/isi.310326
38. arXiv:2409.02683v1 [cs.CV] 4 Sep 2024, https://arxiv.org/pdf/2409.02683
39. Quo Vadis Handwritten Text Generation for Handwritten Text Recognition? - arXiv, https://arxiv.org/html/2508.09936v1